

WESTERN SYDNEY
UNIVERSITY



Module 7

User-defined Functions Overview

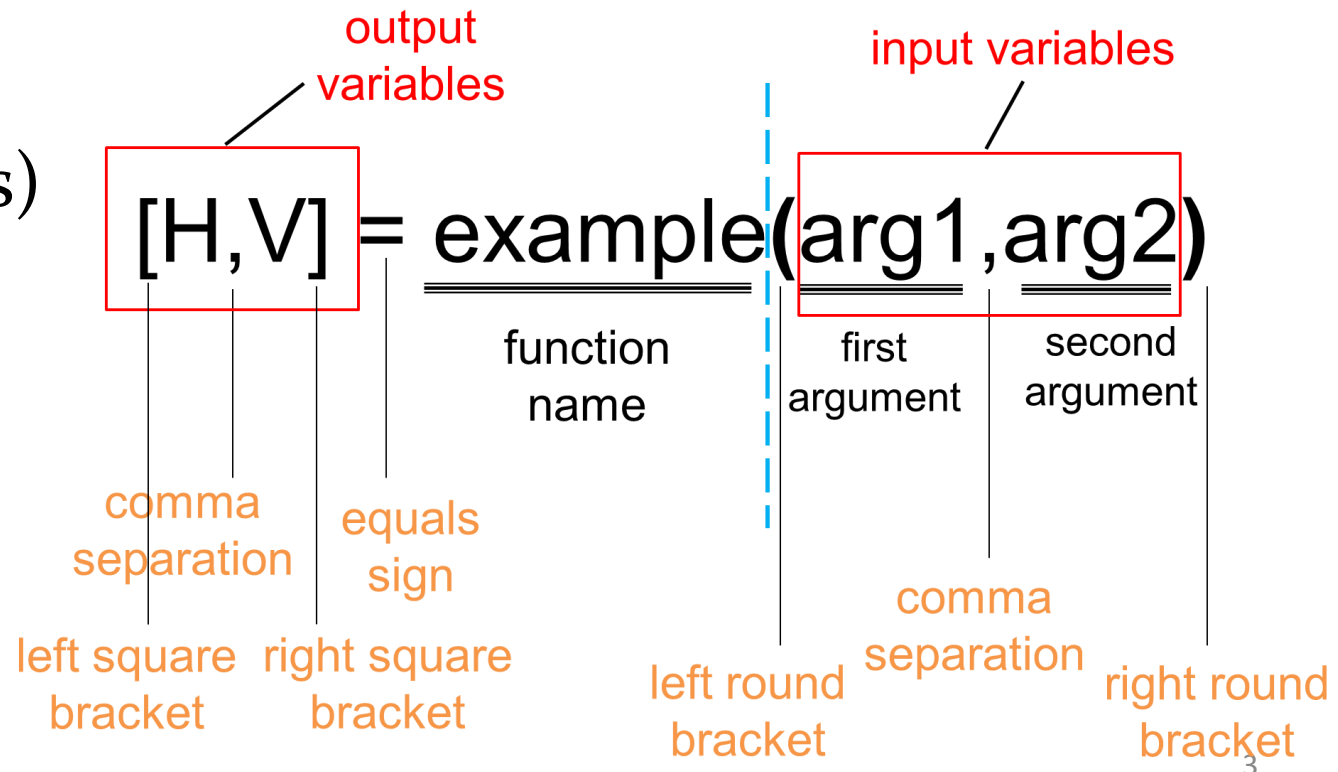
User-defined functions

- User-defined functions are similar to the MATLAB in-built functions.
- They take a certain number of inputs, perform some operation, and give output(s).
- Just as with MATLAB built-in functions, we need to know what they are supposed to do and know that it does it correctly.

Syntax of MATLAB functions

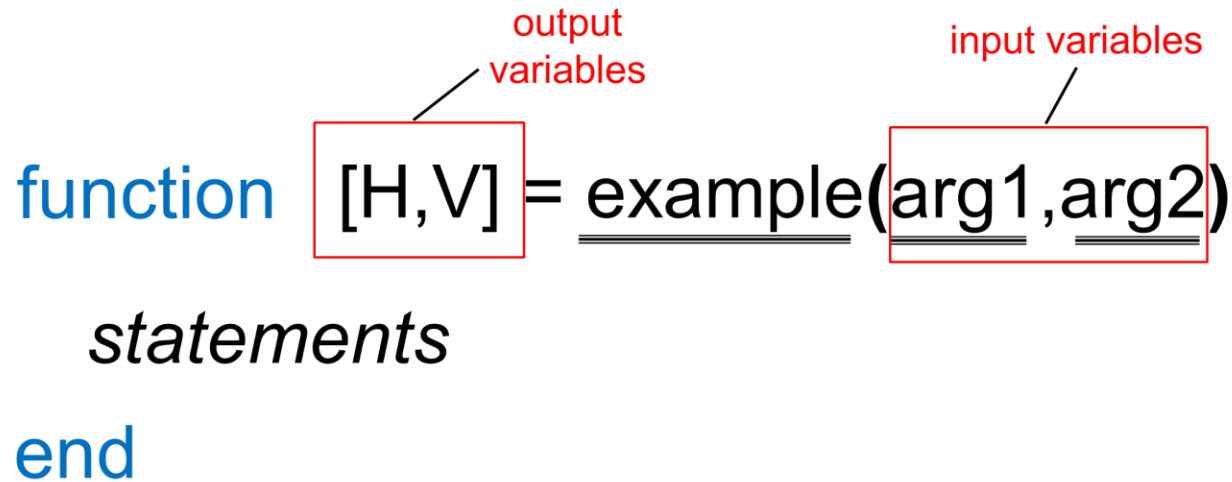
- Both built-in and user defined functions have similar syntax

- Function name
- Input(s)
- Output(s)



Structure of user-defined MATLAB functions

`function` `[H,V]` = `example`(`arg1`,`arg2`)
statements
`end`



The diagram illustrates the syntax of a MATLAB function. The word 'function' is in blue. The output variables '[H,V]' are enclosed in a red box, with a red line pointing to the text 'output variables' above it. The function name 'example' is underlined. The input arguments 'arg1' and 'arg2' are each enclosed in a red box, with a red line pointing to the text 'input variables' above them. The word 'end' is in blue. The word 'statements' is in italics.

- begins with the keyword `function`
- ends with the keyword `end`
- following the function keyword is the normal built-in function structure i.e. `outputs = functionName(inputs)`
- The output and input arguments are comma separated

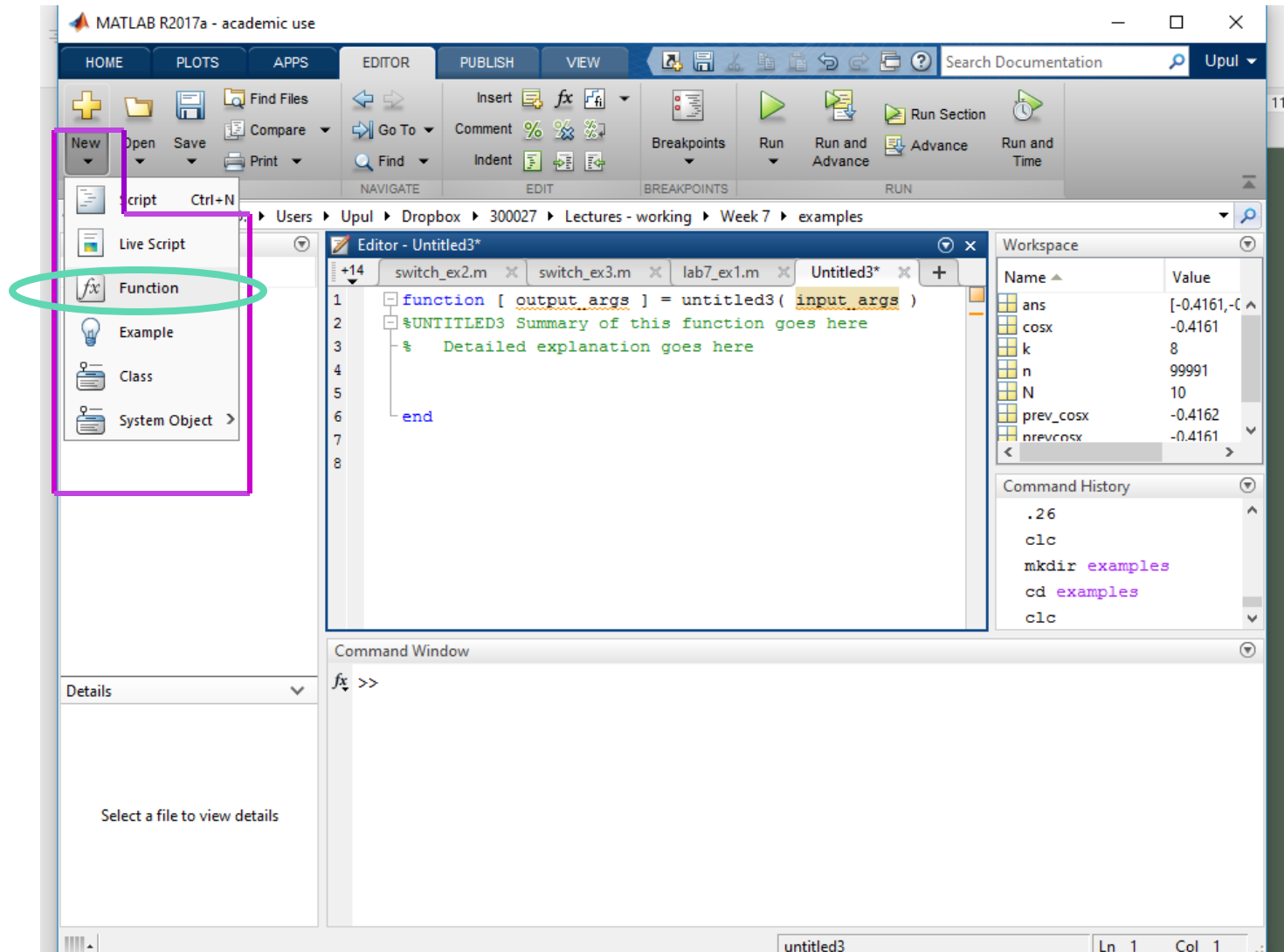
User-defined MATLAB functions

- The user-defined function m-file name should be the same name as the function name.
 - e.g. the user –defined function **myfunction** should be written in the file **myfunction.m**

`myfunction.m`

```
function [oArg1,oArg2]= myfunction(iArg1,iArg2)
    %      statements go here
end
```

Function m-files can be created using the MATLAB Editor



Notes on functions

- The **convention for naming functions** is the same as **variables naming rules**. A function name must start with a letter, should be meaningful, should not use the name of an existing function, and should not be excessively long.
- It is important that you give **meaningful variable names** to variables inside a function that you write, so that you and others can understand what the function does.
- **Comments** become extremely important in functions. Comments help both you and anyone who might use the function to understand what it does.

Example of a function M-file

The first line has the keyword `function` and the function name

```
function [volume] = volume_sphere(radius)
```

```
volume = 4/3*pi*radius^3;
```

It is a good idea to always suppress intermediate results

```
end
```

The function is saved in `volume_sphere.m` file.

Calling a function

- A function call is simply a statement which uses the function (not any different to how you would be doing this already)
- User-defined functions are accessed in the same way as any MATLAB built-in functions.

```
vol = volume_sphere(3)
```

- The function file must be saved in the current directory – if not the directory where it is saved should be in the MATLAB **path**.

See MATLAB documentation on **path** command.

Input/output & functions

- Several kinds of functions can be created with different combinations of input and/or output:
 - Functions with input and output arguments
 - Functions with input arguments and no output arguments
 - Functions with output arguments and no input arguments
 - Functions with neither input arguments nor output arguments

A function with multiple inputs and outputs

```
function [h, A] =right_triangle(a, b)
%right_triangle calculates hypotenuse (h) and the area (A) of
% a right-angled triangle given the two sides a and b
%
% calling sequence
% [h, A] = right_triangle(a, b)
h= sqrt(a^2 + b^2);
A=1/2*a*b;
end
```

```
>> [hypotenuse, area]=right_triangle(3,4)
hypotenuse =
    5
area =
    6
```

Returning an error from a function

```
function [h, A] =right_triangle(a, b)
%right_triangle calculates hypotenuse (h) and the area (A) of
% a right-angled triangle given the two sides a and b
%
% calling sequence
% [h, A] = right_triangle(a, b)
if (a<=0) | (b<=0)
    error('incorrect dimensions for a right-angled triangle');
end
h= sqrt(a^2 + b^2);
A=1/2*a*b;
end
```

```
>> [hypotenuse, area]=right_triangle(3,0)
Error using right_triangle (line 8)
incorrect dimensions for a right-angled triangle
```

Functions with no input

- If you need to access some constant value a number of times, instead of hard-coding the constant value every time it is needed, we can hard-code it once into a function that has no input.
- When the constant is needed, its value is called via the function.
- As an example, if we needed, for whatever reason, the mass of the earth (in kg) multiple times, we could write a function to store it as follows:

```
function m=moe()  
    m=5.976e24; % kg  
end
```

We can call the function as `moe()` or just `moe` in this case (when no inputs).

```
>> moe()  
ans =  
    5.9760e+24  
>> mass=moe()  
mass =  
    5.9760e+24  
>> mass=moe  
mass =  
    5.9760e+24
```

Functions with no output

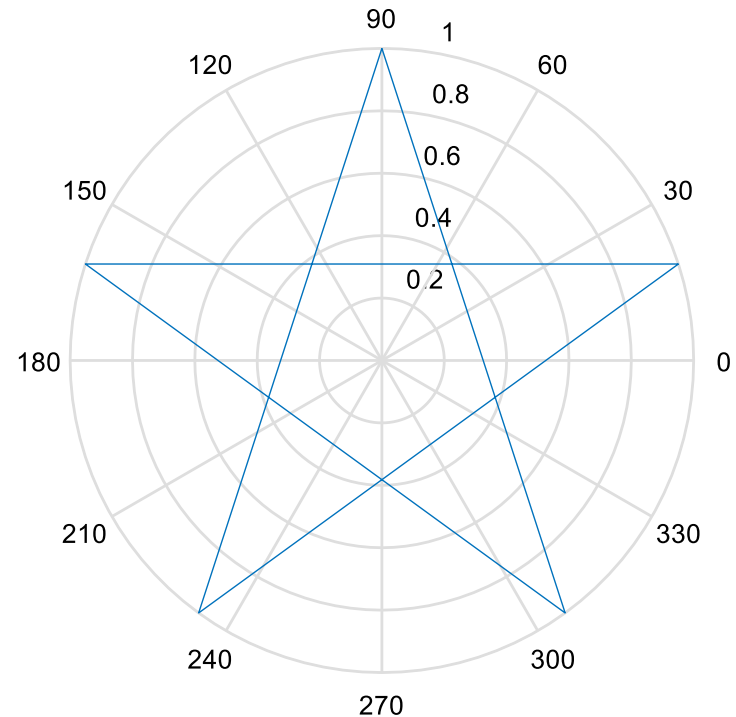
- Functions with no output can be used for many purposes, such as to plot figures from a set of input data or print input information to the command window (output here implies results from operations on input; not just displays).
- The built-in MATLAB function `tic` has no output. It starts a timer going for use with another built-in MATLAB function, `toc`.

Functions with no input or output

- A function with no input or output consists of some hard-coded information that is to be used in a specific procedure.
- An example of this is a function that plots a specific figure. Plotting the figure is the only thing the function does; it does not have any input or operations that lead to computed values for output.

A function with no input or output

```
function [] = star()  
theta = pi/2:0.8*pi:4.8*pi;  
r = [1 1 1 1 1 1];  
polar(theta, r);  
end
```



Although the user-defined function **star** generates a plot, in the strictest sense it does not have any output. If you try to evaluate the expression **A = star** in MATLAB, you will get an error, because **star** has no output.